Intelligent Parallel Scaling for High-Performance Workloads

Students: Hugo Wan (twan012), Adil Mohiuddin (amohi010), Sachin Chopra (schop021)

1. Introduction and Background

Parallel computing is widely used in modern systems to improve performance, especially with multi-core processors. However, getting good performance is not always straightforward. Issues like synchronization overhead, uneven workload distribution, and resource contention can reduce efficiency.

In shared-memory systems, threads need to coordinate through a common memory space, which often requires synchronization. This can introduce overhead and limit scalability. Because of this, simply increasing the number of threads does not always lead to better performance.

In this project, we explore whether **dynamically adjusting the number of threads** can improve performance compared to using a fixed number of threads.

2. Project Goal and Scope

The main goal of this project is to design and evaluate a parallel system that can adjust its thread count based on workload conditions.

We aim to answer the following questions:

- How does the number of threads affect performance?
- Can dynamic thread scaling perform better than static configurations?
- What trade-offs exist between performance improvement and overhead?

3. Approach

We will implement parallel programs using a shared-memory model such as Pthreads (and possibly OpenMP). The workloads will be computationally intensive and easy to parallelize, such as matrix multiplication or numerical computations.

We will compare three approaches:

1. **Static Thread Allocation**
   A fixed number of threads is used throughout execution (baseline)
2. **Dynamic Thread Scaling**
   The number of threads is adjusted based on input size or runtime behavior
3. **Optimized Parallel Execution**
   Improve workload distribution and reduce synchronization overhead

The implementation will follow a **fork-join model**, where threads are created, perform work, and then synchronize.

4. Platform and Tools

- Programming Language: C/C++
- Parallel Frameworks: Pthreads (primary), optional OpenMP
- Environment: Linux (local machine or lab servers)
- Tools: `time`, `perf` for measuring performance

5. Evaluation Plan

We will evaluate each approach using:

- Execution time (runtime)
- Speedup compared to sequential execution
- Parallel efficiency
- CPU utilization

For experiments, we will:

- Vary the number of threads (e.g., 1, 2, 4, 8, etc.)
- Test different input sizes (small to large workloads)
- Compare static and dynamic approaches

We will also generate plots such as runtime vs. thread count and speedup vs. thread count. In addition, we will analyze the overhead introduced by dynamic scaling and how synchronization affects performance.

6. Expected Outcome

We expect that static thread allocation may work well for certain workloads but will not adapt well when workload size changes. Dynamic thread scaling may improve performance for larger workloads, although it may introduce some overhead.

Overall, this project will help us better understand how thread management affects performance and what trade-offs exist when designing parallel systems.